

商店销量时间序列预测项目总结

1. 项目目标

本项目的任务是完成 Kaggle 比赛 Store Sales - Time Series Forecasting 的销量预测。

比赛要求根据历史销售、门店信息、油价、节假日、促销和交易量等数据，预测未来 15 天的门店-商品系列销量。

评价指标是 RMSLE，因此模型不仅要尽量减小绝对误差，也要对低销量样本保持稳定。

项目的最终目标可以概括为两点：

1. 做出一套可以稳定复现的完整代码；
2. 在 Kaggle 公榜上拿到尽可能低的分数。

2. 初始阶段：先把问题跑通

项目刚开始时，我先做了三件事：

1. 阅读比赛数据结构，确认输入文件包含
train.csv、test.csv、stores.csv、oil.csv、holidays_events.csv、transactions.csv。
2. 搭建最小可运行的基线流程。
3. 先用一个简单模型把提交文件跑出来，确认整个数据管道没有断点。

这个阶段的重点不是追求高分，而是确认：

- 训练集和测试集能顺利读取；
- 日期字段能正确解析；
- 特征合并不会出错；
- 提交文件能按 Kaggle 格式生成。

最早的基线主要采用了：

- 日期特征；
- 门店和商品信息；
- oil 和 holidays 特征；
- sales 的 lag / rolling 特征；

- LightGBM 回归。

这一步的意义是建立一个可靠起点，后面的所有迭代都围绕这条主线展开。

3. 第一轮优化：把数据特征做实

在能跑通基线后，我开始补充更适合时间序列问题的特征。

3.1 日期特征

加入了以下时间变量：

- 年、月、日
- 星期几
- 一年中的第几天
- 是否周末
- 是否月初、月末
- 周期性正余弦编码

这些特征的作用是帮助模型识别日历周期和节律变化。

3.2 节假日和特殊日期

增加了节假日相关信息，包括：

- 全国、区域、地方节假日
- 节假日类型
- 是否工作日
- 是否桥接日

同时补充了两类业务日期：

- 发薪日相关特征
- 地震事件相关特征

这样做的原因是：这类事件通常会直接影响消费行为，特别是促销期、节日和特殊事件前后，销量波动明显。

3.3 油价特征

oil.csv 存在缺失值和非连续日期，我对油价做了插值和补全，并构造了：

- 当前油价
- 油价一阶差分
- 油价涨跌幅
- 高油价标记

这样可以让模型更容易捕捉宏观经济波动对消费的影响。

3.4 零销量规则

在训练数据里，部分 store_nbr + family 组合长期不销售任何商品。这类组合在测试中通常也应保持接近 0，因此我加入了零销量规则：

- 如果某个门店-商品系列历史总销量为 0；
- 且当前没有促销；
- 则将预测值强制为 0。

这一步对最终分数有明显帮助，也是后面版本一直保留的稳定规则。

4. 第二轮优化：加入时序核心特征

时间序列预测里，真正有效的通常还是历史销量本身。

因此我进一步加入了两类核心特征：

4.1 Lag 特征

包括：

- lag_1
- lag_7
- lag_14
- lag_28

这些特征分别代表前一天、前一周、前两周和前四周的销量。

4.2 Rolling 特征

包括：

- 7 日均值
- 28 日均值
- 7 日标准差
- 28 日标准差

Rolling 特征能帮助模型理解最近一段时间的局部趋势和波动幅度。

4.3 Transactions 特征的尝试与取舍

我也尝试过把 transactions 及其 lag / rolling 特征加入模型，因为它对销量有一定解释能力。

但后面在递推预测和测试阶段发现：

- 训练集里能拿到完整交易量；
- 测试集未来日期没有真实交易量；
- 这会导致训练/测试特征不一致。

因此最终版本中，把 transactions 相关特征从模型输入中剔除，改为保留为数据分析参考，而不直接参与最终预测。

5. 模型探索过程

5.1 LightGBM 作为主模型

LightGBM 一直是项目里最稳定的主模型，原因有三个：

1. 对表格型特征非常友好；
2. 对非线性关系和交互项处理较强；
3. 在中等规模数据上训练速度可接受。

早期基线和中间版本大多以 LightGBM 为主。

5.2 深度趋势/残差思路的尝试

我中间尝试过更复杂的发展趋势-残差结构，希望通过趋势模型捕捉长期变化，再用树模型拟合残差。

这条路线的问题在于：

- 结构变复杂；
- 训练和验证不稳定；
- 调参成本高；
- 对工程一致性要求很高。

后来我放弃了这条路线，回到更稳的表格模型方案。

5.3 LightGBM + XGBoost 集成

最终比较稳的一版是：

- LightGBM 做主模型；
- XGBoost 做辅助模型；
- 根据验证分数计算融合权重；
- 如果第二模型明显弱，就自动降低其权重。

这套组合的优点是：

- 结构清晰；
- 容易复现；
- 对代码维护友好；
- 比单模型更稳。

6. 最终关键改动：真正的递推预测

这是整个项目里最重要的一步。

6.1 问题背景

测试集是连续 15 天的未来销量预测。

如果直接把测试集一次性塞进模型，那么：

- 第 2 天以后的 lag 特征没有真实值；
- rolling 特征无法连续更新；
- 预测过程和真实提交场景不一致。

6.2 递推方案

最终我把预测流程改成了 day-by-day recursive forecast:

1. 先预测第 1 天；
2. 将第 1 天预测结果写回历史；
3. 用“真实历史 + 预测历史”生成第 2 天特征；
4. 继续预测第 2 天；
5. 重复直到 15 天结束。

这样做后，模型的验证方式和 Kaggle 提交方式保持一致，公榜表现也明显提升。

6.3 递推版的价值

递推机制带来的核心收益是：

- lag 特征连续；
- rolling 特征连续；
- 验证更接近真实测试；
- 提交分数更稳定。

这一步是最终从较高分区间显著降分的关键。

7. 最终版本的特征体系

最终模型保留的特征大致分为五类：

7.1 时间特征

- 年、月、日、星期
- 一年中的第几天
- 是否周末
- 周期性正余弦编码
- 发薪日相关特征

7.2 外部业务特征

- 门店信息
- 商品家族信息

- 门店类型、城市、州
- 油价及其派生特征
- 节假日及事件特征
- 地震相关特征

7.3 历史销量特征

- 销量 lag
- 销量 rolling mean
- 销量 rolling std

7.4 静态聚合特征

基于训练集统计得到：

- store-level mean/std
- family-level mean/std
- store-family mean/std
- family-by-weekday mean/std
- type / city / state 的均值统计
- family-by-month、store-by-month 的均值统计

7.5 规则特征

- 零销量门店-商品组合强制置零

8. 最终代码架构

最终保留下来的主结构是：

```
src/
├── data/
│   └── load_data.py
├── ensemble_v1/
│   ├── config.py
│   ├── ensemble.py
│   ├── features.py
│   ├── models.py
│   └── pipeline.py
├── features/
│   └── make_features.py
├── models/
│   └── train_lgbm.py
└── utils/
    └── metrics.py

scripts/
└── run_ensemble_v1.py
```

各文件职责如下：

- load_data.py：读取和合并原始 Kaggle 数据；
- make_features.py：日期、lag、rolling、统计聚合特征；
- ensemble_v1/features.py：最终特征拼装、零销量规则；
- ensemble_v1/models.py：LightGBM / XGBoost 训练与预测；
- ensemble_v1/pipeline.py：训练、验证、递推预测、提交文件生成；
- run_ensemble_v1.py：命令行入口；
- metrics.py：RMSLE 计算。

9. 结果记录

9.1 主要迭代结果

阶段	版本	核心思路	结果
初始基线	baseline	LightGBM + 基础时序特征	公榜约 2.67
中间探索	ensemble_v1 初版	多模型融合、更多特征	公榜约 2.44

阶段	版本	核心思路	结果
递推改造后	ensemble_v1 recursive	day-by-day 递推预测	明显提升
最终版本	ensemble_v1_full.csv	递推 + 稳定特征 + 融合	0.42660

9.2 最终提交

- 文件名： submissions/ensemble_v1_full.csv
- 提交时间： 2026-05-01 14:50:06
- Kaggle 公榜分数： 0.42660

10. 运行方式

安装依赖：

```
pip install -r requirements.txt
```

快速验证：

```
python scripts/run_ensemble_v1.py --mode fast
```

最终训练与提交：

```
python scripts/run_ensemble_v1.py --mode full
```

11. 项目总结

这个项目从一开始的简单基线，逐步演变成一套可以稳定复现的时间序列预测工程。

整个过程里最关键的经验是：

1. 先把基线做稳，再一点点加特征；
2. 特征越多不一定越好，训练/测试一致性更重要；
3. 时间序列问题里，递推预测常常比一次性批量预测更贴近真实场景；

4. 规则特征、业务特征和历史特征的组合，往往比复杂结构更有效。

最终版本不是最花哨的，但它是最稳定、最能交付、也最适合作为课程期末项目的版本。